

ATENÇÃO: Esta é a questão problema

Criar um sistema de gerenciamento de pedidos online escalonável: O objetivo é que os 'engenheiros' projetem a arquitetura de um sistema de pedidos online, como um aplicativo de delivery de comida, com foco na capacidade de lidar com um grande volume de tráfego e transações.

Requisitos Funcionais e Não-Funcionais

1. Funcionais principais

- Catálogo de produtos: Visualização e busca de produtos/itens.
- Cesta de compras: Adicionar, remover e atualizar itens antes da finalização.
- Processamento de pedidos: Criação, validação, e gerenciamento do ciclo de vida de um pedido (pendente, confirmado, em preparo, em entrega, concluído).
- Autenticação de usuário: Cadastro e login de clientes.

2. Não-Funcionais (foco na escalabilidade)

- Escalabilidade (alta disponibilidade): O sistema deve ser capaz de suportar picos de 10.000 pedidos por minuto.
- Latência: A finalização do pedido (checkout) deve ser concluída em menos de 2 segundos.
- Consistência: A informação do estoque deve ser a mais próxima do tempo real possível, mas a prioridade é o alto volume de pedidos.

Fases da análise

1. Definição da arquitetura e camadas

- Desenho da arquitetura: Decidir sobre a arquitetura geral (diagrama de blocos) que suporte a escalabilidade.
 - Perguntas-chave:
 - Como o sistema será dividido? (Ex: micro serviços, arquitetura orientada a eventos (EDA), monolito modularizado)
 - Quais camadas de software (apresentação, controle, persistência, dados) serão utilizadas em cada componente?
 - Onde e como o sistema usará *load balancers* e *autoscaling*?
- Modelagem de componentes chave: Definição dos principais serviços (ex: serviço de catálogo, serviço de pedidos, serviço de pagamento, etc.).

2. Aplicação de Coesão, Acoplamento e Consciência

Para um serviço específico (ex: o serviço de pedidos), detalhar a implementação interna.

- Coesão (Alta): Como garantir que o serviço de pedidos tenha alta

coesão? (ou seja, todas as suas classes e métodos trabalham em conjunto para cumprir apenas a responsabilidade de gerenciar o ciclo de vida de um pedido, sem se intrometer no pagamento ou estoque).

- Acoplamento (Baixo): Como o serviço de pedidos se comunica com outros serviços (ex: serviço de estoque e serviço de pagamento) para garantir um baixo acoplamento?
 - Exemplo para discussão: uso de mensageria assíncrona (filas/tópicos) em vez de chamadas síncronas HTTP diretas.
- Consciência: Identificar e definir um tipo de consciência entre duas classes internas no serviço de pedidos (ex: uma classe pedido e uma classe repositóriodepedido).
 - *Exemplo:* consciência de nome (se renomear uma, tem que renomear a outra) ou consciência de posição (se a ordem dos parâmetros mudar na interface, precisa mudar na chamada).

3. Solução de escalabilidade específica (derivados)

- Separação de leitura e escrita (CQRS): Como o sistema pode separar as operações de alta leitura (ex: consulta ao catálogo, *feed* de pedidos de um restaurante) das operações de alta escrita (ex: o *checkout* final do pedido)?
 - Sugestão: Usar *Read Replicas* ou o padrão CQRS (Command Query Responsibility Segregation), utilizando um tipo de banco de dados para escrita (ex: SQL) e outro, otimizado para leitura (ex: NoSQL cacheado ou *search engine*), para consultas.
- Sharding/Particionamento: Como o banco de dados de pedidos será dividido (*sharding* ou particionamento) para suportar o volume? Qual seria a chave de partição (Ex: ID do usuário, ID da loja, data)?

O que um engenheiro deve entregar?

Deve entregar um documento de design de sistema contendo:

1. Diagrama de arquitetura de alto nível (com serviços, *load balancers*, filas, bancos de dados).
2. Descrição da abordagem de escalabilidade (mencionando derivados como microserviços, *sharding*, cache).
3. Discussão detalhada sobre a aplicação de coesão, acoplamento e consciência em pelo menos um serviço da arquitetura.

ATENÇÃO: Esta é a solução proposta

Uma solução utilizando a arquitetura de microserviços e a arquitetura orientada a eventos (eda), que são os padrões mais adequados para atingir a alta escalabilidade exigida (10.000 pedidos/min).

Resolução sugerida: sistema de pedidos escalonável

1. Arquitetura de alto nível (escalabilidade e derivados)

O sistema deve ser dividido em microsserviços, cada um com sua própria responsabilidade e banco de dados isolado (princípio de responsabilidade única e baixo acoplamento).

Microsserviço	Responsabilidade principal	Estratégia de escalabilidade
Gateway (API Gateway)	Roteamento, autenticação, rate limiting.	Load Balancing, Autoscaling (Replicação Horizontal).
Catálogo/Produtos	Visualização e busca de produtos.	Alto Cache (Redis/Memcached) e <i>Read Replicas</i> no DB. Padrão CQRS natural (muita leitura, pouca escrita).
Cesta de Compras	Gerenciamento de itens temporários do usuário.	Banco de dados NoSQL (ex: DynamoDB, MongoDB) ou Cache para latência mínima e escalabilidade horizontal.
Pedidos (Orders)	Criação e gerenciamento do ciclo de vida do pedido.	EDA (Assíncrono) e Sharding/Particionamento do DB por ID do Cliente ou ID da Loja.
Pagamentos	Processamento e comunicação com <i>gateways</i> externos.	Isolamento (Bulkhead) e processamento Assíncrono para resiliência.
Estoque/Inventário	Checagem e decremento do estoque.	Otimização para escrita rápida e mensageria para atualização.

O papel do event broker (fila de mensagens)

Para atingir baixo acoplamento e alta resiliência (isolamento de falhas), a comunicação entre os serviços deve ser primariamente assíncrona via um event broker (ex: kafka ou rabbitmq).

Fluxo de exemplo (o desafio da escrita):

1. O serviço de pedidos recebe o pedido (alta taxa de escrita).

2. Ele salva o status inicial (pendente) em seu próprio BD.
3. Ele publica um evento: orderplaced no tópico do event broker.
4. O serviço de pagamentos e o serviço de estoque (consumidores) escutam esse evento e iniciam suas operações de forma independente.
 - *Resultado*: o serviço de pedidos não espera por nada, garantindo a baixa latência na finalização do pedido, focando apenas em sua responsabilidade (alta coesão).

2. Aplicação de Coesão, Acoplamento e Consciência

Coesão (alta)

- Implementação: o serviço de pedidos deve ser composto apenas por classes e métodos que tratam do ciclo de vida do pedido (criar, atualizar status, buscar por id).
- Exemplo: a classe pedidoservice terá métodos como criarnovopedido(), atualizastatus(id, novostatus), e buscaporid(id). Não deve ter métodos como envianotificacaoemail() ou processapagamento(). Essas são responsabilidades de outros serviços, garantindo a coesão funcional.
- Camadas de software: internamente, o serviço de pedidos deve usar a arquitetura em camadas:
 - Apresentação: pedidocontroller (recebe a requisição http e chama o serviço).
 - Negócio/aplicação: pedidoservice (aplica regras de negócio, como calcular impostos/frete).
 - Dados: pedidorepository (lida com a persistência no bd).

Acoplamento (baixo)

O principal objetivo é desacoplar o serviço de pedidos de outros serviços.

- Acoplamento indesejado (forte): se o serviço de pedidos fizesse uma chamada http síncrona direta para o serviço de pagamento. Se o pagamento cair, o pedido falha.
- Solução (fraco/baixo acoplamento):
 - Mensageria: como visto acima, o serviço de pedidos publica o evento (orderplaced) e segue em frente. Ele não sabe nem se importa quem irá processar esse evento, apenas que ele foi emitido.
 - Interfaces/abstrações: uso de injeção de dependência (di) e interfaces dentro do código interno do serviço para não depender de implementações concretas (ex: ipedidorepository em vez de depender da classe mysqlpedidorepository).

Consciência (consciente)

A consciência é usada para medir a qualidade do acoplamento em um nível mais granular.

- Exemplo de consciência desejável (fraca): consciência de nome
 - Classe pedido: possui o campo status.
 - Classe pedidorepository: possui o método atualizarstatus(id, status).
 - Se o desenvolvedor renomear o campo status na classe pedido, ele terá que renomear o parâmetro status no método do repositório. É um acoplamento leve, fácil de detectar e corrigir, mantendo o código consistente.
- Exemplo de consciência a ser evitada (forte): consciência de posição
 - Se tivéssemos uma função: criarpedido(nomecliente, valor, data)
 - E na chamada, por engano, fosse: criarpedido(valor, nomecliente, data). O código compilaria, mas daria erro em tempo de execução.
 - Solução: usar um objeto de transferência de dados (dto) nomeado, como criarpedido(pedidodto), para evitar a consciência de posição, forçando a consciência de nome, que é mais fácil de gerenciar e refatorar.